

使用燃原 Enflame S60 运行 ERNIE-Image 模型

本文档整理了 ERNIE-Image 在燃原 Enflame S60 云实例环境下的部署过程、服务化方案、Web UI 方案，以及已完成的一轮性能测试结果。

当前结论可表述为：

ERNIE-Image 已在 Enflame S60 环境完成主推理链路打通，并已验证单次推理、API 服务、Gradio Web UI 和基础性能测试能力。当前结论属于功能级适配成功 / POC 跑通，尚不等同于“正式生产级适配完成”。

参考资料

- ERNIE-Image 开发分支：
<https://github.com/HsiaWinter/diffusers>
- Diffusers 官方合入 PR：
<https://github.com/huggingface/diffusers/pull/13432>

1. 概述

ERNIE-Image 已在 Enflame S60 云实例环境完成部署验证。本文档给出：

- 环境要求
- 模型与代码准备
- 安装与部署方法
- 单次推理验证
- API 服务与 Web UI 部署
- 常见兼容性问题
- 性能测试结果与工程评估

本次部署基于 `diffusers` 的 `add-ernie-image` 分支完成，结合实例内已有 `torch_gcu` / TOPS 运行时，在 GCU 上完成图像生成。

实际部署过程中，无需改动模型权重格式，主要工作集中在：

- `diffusers` 分支安装
- `transformers` 版本兼容
- GCU 设备适配
- 数值稳定性调整
- 服务化与基准测试脚本落地

2. 环境准备

2.1 基础要求

- Linux x86_64
- Python >= 3.10

- Enflame S60 GCU
- 已安装 TOPS / [torch_gcu](#)
- 建议使用独立 Python 环境
- 建议准备：
- 模型权重包 [ERNIE-Image.tar](#)
- [/data](#) 目录用于模型、输出、日志、基准测试结果存储

2.2 已验证环境

项目	版本 / 信息
Device	Enflame S60
OS	Ubuntu 24.04.3 LTS
Python	3.12.3
Torch	2.8.0+cpu
torch_gcu	2.8.0+3.6.0.23
Diffusers	0.38.0.dev0 (HsiaWinter/diffusers add-ernie-image)
Transformers	5.5.1
Gradio	5.50.0
FastAPI	0.118.0
Uvicorn	0.40.0
GCU 数量	1
GCU 显存	47990 MB

说明：

- 当前实例不是 NVIDIA / CUDA 环境，而是 Enflame S60 GCU 环境。
- 实例内已安装 [torch_gcu](#)、TOPS 运行时和相关库。
- 实际推理已成功生成图片，并通过 API 与 Gradio Web UI 验证。

3. 模型与代码准备

3.1 模型权重

模型权重已放置在服务器 [/data](#) 目录下，源文件名包含下载鉴权参数，例如：

```
/data/ERNIE-Image.tar?authorization=...
```

该文件可直接使用，无需强制重命名。

3.2 推理代码

ERNIE-Image 当前使用自定义 [diffusers](#) 分支：

```
git clone https://github.com/HsiaWinter/diffusers.git
git checkout add-ernie-image
```

实际部署路径：

```
/data/ernie-image-deploy/src/diffusers
```

4. 环境安装

建议使用独立 Python 环境，并继承系统已有 GCU 相关包。

4.1 创建环境

```
python3 -m venv --system-site-packages /data/ernie-image-deploy/venv  
source /data/ernie-image-deploy/venv/bin/activate
```

4.2 安装基础依赖

```
pip install --upgrade pip setuptools wheel  
pip install fastapi 'uvicorn[standard]' python-multipart
```

4.3 安装 ERNIE-Image 对应 diffusers

```
cd /data/ernie-image-deploy/src/diffusers  
pip install --no-deps -e .
```

4.4 安装兼容版 transformers

ERNIE-Image 权重中的 `text_encoder` / `pe` 配置依赖：

- `mistral3`
- `ministral3`

低版本 `transformers` 无法识别，会报：

```
KeyError: 'ministral3'
```

因此本次部署升级为：

```
pip install --upgrade 'transformers==5.5.1'
```

5. 模型解压与目录组织

建议目录结构如下：

```
/data/ernie-image-deploy/
├── model/
│   └── ERNIE-Image/
├── outputs/
├── run/
├── src/
│   └── diffusers/
├── bench/
│   ├── results/
│   └── extra/
├── venv/
├── common.py
├── 01_single_run.py
├── 02_api.py
├── 03_start.sh
├── 04_selftest.sh
├── 05_gradio.py
└── 06_start_gradio.sh
```

解压模型：

```
mkdir -p /data/ernie-image-deploy/model
tar -xf "/data/ERNIE-Image.tar?authorization=..." -C /data/ernie-image-deploy/model
```

解压后模型目录应类似：

```
ERNIE-Image/
├── model_index.json
├── transformer/
├── vae/
├── scheduler/
├── tokenizer/
├── text_encoder/
├── pe/
└── pe_tokenizer/
```

6. 环境验证

安装完成后，可通过以下代码验证环境：

```
import torch
import torch_gcu
from diffusers import ErnieImagePipeline

print('torch:', torch.__version__)
print('has torch.gcu:', hasattr(torch, 'gcu'))
print('gcu device count:', torch.gcu.device_count())
print('pipeline:', ErnieImagePipeline)
```

本次实际验证结果：

- `torch_gcu` 可导入
- `torch.gcu.device_count() == 1`
- `ErnieImagePipeline` 可导入
- 推理链路可在 `gcu:0` 上运行

7. 单次推理验证

建议先做单次推理，再启动服务。

7.1 推理方式

本次最终采用：

- 设备：gcu:0
- dtype: bfloat16
- use_pe=True
- 512 / 1024 分辨率均已验证

单次推理脚本：

```
cd /data/ernie-image-deploy
source /data/ernie-image-deploy/venv/bin/activate
export HF_HOME=/data/huggingface_home
export ENFLAME_PT_GELU_ERF_ENABLE=true

python 01_single_run.py \
  --prompt '一只黑白相间的中华田园犬' \
  --height 512 \
  --width 512 \
  --num-inference-steps 8 \
  --guidance-scale 5.0 \
  --use-pe
```

7.2 输出检查

本次实际验证结果：

- 输出文件存在
- 文件格式：PNG
- 分辨率：512x512
- 文件大小：约 446 KB
- 像素统计正常，不是黑图 / 空图

示例输出：

```
/data/ernie-image-deploy/outputs/20260409-103026-0150b60a.png
```

说明：

- 在 GCU 环境中，float16 版本首次生成曾出现全黑图。
- 切换到 bfloat16 并设置 ENFLAME_PT_GELU_ERF_ENABLE=true 后恢复正常。

8. 服务化部署

8.1 启动 API 服务

当前服务基于 FastAPI + Uvicorn：

```
cd /data/ernie-image-deploy
PORT=8188 ./03_start.sh
```

默认监听：

```
0.0.0.0:8188
```

8.2 健康检查

```
curl http://127.0.0.1:8188/health
```

返回示例：

```
{
  "status": "ok",
  "model_dir": "/data/ernie-image-deploy/model/ERNIE-Image",
  "device": "gcu",
  "torch_device": "gcu:0",
  "dtype": "bfloat16",
  "load_seconds": 28.51,
  "outputs_dir": "/data/ernie-image-deploy/outputs"
}
```

8.3 生成接口

```
curl -X POST http://127.0.0.1:8188/generate \
-H "Content-Type: application/json" \
-d '{
  "prompt": "一只黑白相间的中华田园犬",
  "height": 512,
  "width": 512,
  "num_inference_steps": 8,
  "guidance_scale": 5.0,
  "use_pe": true
}'
```

返回示例（节选）：

```
{
  "prompt": "一只黑白相间的中华田园犬",
  "elapsed_seconds": 20.11,
  "peak_memory_allocated_gb": 29.297,
  "peak_memory_reserved_gb": 30.133,
  "image_path": "/data/ernie-image-deploy/outputs/20260409-103434-b5ae78f1.png"
}
```

8.4 服务状态文件

- PID 文件：

[/data/ernie-image-deploy/run/service.pid](#)

- 服务日志：

[/data/ernie-image-deploy/run/service.log](#)

9. Web UI 部署

本次在现有 API 之上额外部署了 Gradio Web UI，以避免重复加载模型。

9.1 启动方式

```
cd /data/ernie-image-deploy
GRADIO_PORT=7860 BACKEND_PORT=8188 ./06_start_gradio.sh
```

9.2 访问地址

```
http://127.0.0.1:7860
```

9.3 UI 功能

- Prompt 输入
- Height / Width 选择 (512 / 768 / 1024)
- Steps / Guidance 调整
- Seed 输入
- `use_pe` 开关
- 示例 Prompt
- 图片预览
- Revised Prompt 展示
- 元数据展示

9.4 SSH 隧道访问

API:

```
ssh -CNg -L 8188:127.0.0.1:8188 root+vm-xZLnOjJMBgXs3GCe@106.13.250.54 -p 32222
```

Gradio:

```
ssh -CNg -L 7860:127.0.0.1:7860 root+vm-xZLnOjJMBgXs3GCe@106.13.250.54 -p 32222
```

10. 性能测试

10.1 测试说明

本轮性能测试已在当前 Enflame S60 实例上完成，测试内容涵盖：

- 吞吐测试
- 时延测试
- 显存峰值测试
- 512 / 1024 / 不同步数性能曲线
- 单并发 / 多并发性能对比

测试对象：

- 当前已部署 FastAPI 服务：<http://127.0.0.1:8188>

采样方式：

- API 端到端 client latency
- 服务返回的 `elapsed_seconds`
- 服务进程内 `torch.gcu.max_memory_allocated/max_memory_reserved`

测试脚本：

- [/data/ernie-image-deploy/bench/run_benchmark.py](#)

- 并发补充测试：本地通过 SSH 隧道执行容错版并发压测

说明：

- 本轮测试是在当前服务实现上完成的
- 当前服务为单进程共享 pipeline
- 多并发结果反映的是现有服务端到端行为，不代表 GCU 真并行理论极限

10.2 单图性能矩阵

分辨率	Steps	平均时延(s)	吞吐(img/s)	峰值显存 Alloc(GB)	峰值显存 Reserved(GB)
512	8	16.385	0.0610	29.294	30.133
512	20	26.554	0.0377	29.295	30.133
512	50	44.578	0.0224	29.294	30.133
1024	8	33.232	0.0301	31.081	34.568
1024	20	68.753	0.0145	31.081	34.568
1024	50	149.823	0.0067	31.080	34.568

10.3 单并发 / 多并发对比

测试配置 A : 512x512, 8 steps

并发	请求数	成功数	失败数	成功率	成功请求平均时延(s)	成功吞吐(img/s)
1	1	1	0	100%	17.373	0.0576
2	2	2	0	100%	25.349	0.0659
4	4	2	2	50%	58.706	0.0323

失败样例：

HTTP 500: index 9 is out of bounds for dimension 0 with size 9

测试配置 B : 1024x1024, 20 steps

并发	请求数	成功数	失败数	成功率	成功请求平均时延(s)	成功吞吐(img/s)
1	1	1	0	100%	67.960	0.0147
2	2	0	2	0%	—	0.0000
4	4	4	0	100%	194.801	0.0142

失败样例：

HTTP 500: index 21 is out of bounds for dimension 0 with size 21

10.4 结果分析

吞吐测试

- 512x512, 8 steps 单并发稳定吞吐约为 0.058 ~ 0.061 img/s
- 同配置在并发 2 时吞吐略有提升，但提升有限
- 1024x1024, 20 steps 稳定吞吐约为 0.014 ~ 0.015 img/s
- 对于大图配置，并发 1 → 4 后成功吞吐基本无增长

说明：

当前部署形态下，吞吐瓶颈主要在服务端单进程共享 pipeline，而非客户端压测能力。

时延测试

- 在同一分辨率下，steps 增长与时延近似线性增长
- 在同一步数下，1024 相比 512 时延大约增加 2x ~ 3.4x
- 1024x50 单图平均时延已达到 149.823s/张

显存峰值测试

- 512 配置下，峰值 Alloc 显存约 29.3 GB
- 1024 配置下，峰值 Alloc 显存约 31.1 GB
- 步数变化对峰值显存影响较小，分辨率影响更明显

可认为：

当前实现中，显存峰值主要由分辨率决定，而非由采样步数决定。

512 / 1024 性能曲线

同 steps 下，1024 相比 512 的时延倍率约为：

- 8 steps: $33.232 / 16.385 \approx 2.03x$
- 20 steps: $68.753 / 26.554 \approx 2.59x$
- 50 steps: $149.823 / 44.578 \approx 3.36x$

因此可得：

分辨率从 512 提升到 1024 后，时延成本显著增加，并随更大 steps 呈扩大趋势。

单并发 / 多并发对比

当前服务由于采用单进程共享 pipeline，在多并发下表现出两个特征：

1. 吞吐提升有限，甚至在异常场景下下降
2. 存在线程安全 / 共享状态问题，导致部分并发档位出现 HTTP 500

因此当前实现更适合：

- 单用户交互式 Web UI
- 低并发 API
- 离线批量生成（串行）

不适合直接作为高并发在线图像生成服务。

10.5 稳定性结论

本轮测试已经证明：

- 单并发功能与性能可用
- 多并发不稳定

并发异常的直接表现为：

```
index 9 is out of bounds for dimension 0 with size 9
index 21 is out of bounds for dimension 0 with size 21
```

这说明当前服务在并发访问时，内部某些张量维度或状态被污染，属于典型的共享 pipeline 并发不安全问题。

工程上建议优先方案：

- 增加全局串行锁，先确保稳定
- 如需真并发，再考虑多进程 worker 隔离
- 如需进一步定位，可重点排查 PE / tokenizer / pipeline 内部共享状态

11. 已完成的部署修复项

本次部署过程中，额外完成了以下兼容性修复：

1. xformers 导入保护
 - 系统中存在 CUDA 版 xformers
 - 在 GCU 环境会误尝试加载 libcuda.so.12
 - 已通过 diffusers 层兼容补丁避免导入失败
2. transformers 版本升级
 - 解决 KeyError: 'minstral3'
3. GCU generator 设备修复
 - 修复 Expected a 'gcu' device type for generator but found 'cpu'
4. 数值稳定性修复
 - GCU 默认改用 bfloat16
 - 启用 ENFLAME_PT_GELU_ERF_ENABLE=true
 - 解决首轮推理出现黑图的问题
5. 服务化与 Web UI 落地
 - FastAPI 服务已上线
 - Gradio UI 已上线

12. 结论

本次 ERNIE-Image 在 Enflame S60 环境下的部署与测试，可以得出以下结论：

1. 功能适配已完成
 - 模型已成功加载
 - 单次推理已成功生成图片
 - API 已可用
 - Gradio Web UI 已可用
2. 单并发性能已具备工程参考价值
 - 可用于离线生成、单用户交互、功能验收

3. 多并发稳定性尚未达标

- 当前实现存在并发 500 错误
- 暂不建议直接作为高并发在线服务投入生产

因此，当前更准确的结论是：

ERNIE-Image 已在 Enflame S60 环境完成功能级部署与基础性能验证，具备继续推进工程化优化的基础，但距离生产级并发服务仍需进一步完善。

13. 结果文件

报告文件

- `/Users/lixiaobai/.codex/超级个体项目孵化/ERNIE-Image_Enflame_S60_部署与性能测试报告.md`

远端结果

- 单并发汇总：

`/data/ernie-image-deploy/bench/results/20260409-110810/single_curve_summary.csv`

- 单并发原始数据：

`/data/ernie-image-deploy/bench/results/20260409-110810/single_curve_raw.csv`

本地整理结果

- 并发汇总：

`/tmp/ernie-local-bench/concurrency_tolerant_summary.csv`

- 并发原始数据：

`/tmp/ernie-local-bench/concurrency_tolerant_raw.csv`

- 单并发曲线图：

`/tmp/ernie-local-bench/single_curves.png`

- 并发对比曲线图：

`/tmp/ernie-local-bench/concurrency_curves.png`